Build a multi-app academic workspace dashboard connecting Google Drive, Excel assignments, and Word lecture notes with automated sync

Computer
Build a Python script that generates a structured semester planner across Word syllabus outlines, Excel gradebooks, and Google Drive folders
Computer
Show the database schema for assignments and files
Detail the Excel import and native spreadsheet mapping
Map the API integration for Google Drive sync

I built a downloadable Python **starter** that generates a Word semester outline, Excel assignment/grade-planning workbook, and review-only Google Drive folder plan. I also generated example outputs and verified that the script completed successfully.

The package includes:

- Python planner script.
- Dependency requirements.
- Example assignment CSV.
- Example Word outline.
- Example Excel workbook.
- Google Drive folder-plan JSON.
- Output manifest with checksums.

The script does not connect to Google Drive, create folders, change a database, or deploy a dashboard. Those actions remain separate, authorization-gated integration work.

Below is the database schema, Excel mapping specification, and Google Drive API integration plan to build the full dashboard around it.

1. Dashboard architecture

Use one Semester workspace with connected panels—not separate copies of assignments in each application.

```
Academic workspace dashboard
│
├── Semester overview
```

```
│   ├── Verified deadlines
│   ├── Assignment progress
│   ├── Upcoming study blocks
│   └── Sync problems requiring attention
│
├── Course workspace
│   ├── Assignment list
│   ├── Word/imported lecture notes
│   ├── Native documents
│   ├── Excel/imported workbooks
│   └── Research sources
│
├── Shared services
│   ├── Artifact storage and version history
│   ├── Assignment/deadline authority
│   ├── Permissions and search
│   └── Audit and notifications
│
└── Integration workers
    ├── Excel importer
    ├── Word importer/exporter
    ├── Google Drive change processor
    └── Sync reconciliation and conflict handling
```

The project's inspected architecture already identifies shared files, search, notifications, policy, audit, outbox, and integration components. Reuse and harden those components rather than introducing a parallel platform. Its October 4 snapshot also reports incomplete adoption and security/AI-policy blockers; that is dated evidence, not verification of today's deployment. [1]

Dashboard cards

Card	Data shown	Primary action
Due next	Applicable deadline, source, verification state	Open assignment
Continue working	Artifact, latest saved version, related assignment	Resume editor
Course files	Native/imported/external status	Open or import
Grade planning	Student-entered results and calculation limitations	Open workbook
Sync health	Last success, pending changes, conflicts	Review issue
Needs verification	Extracted deadlines and source discrepancies	Confirm against source

A synced file, completed draft, exported deliverable, and officially submitted assignment must remain distinct states.

2. Run the Python planner

Install dependencies:

```
python -m pip install -r requirements.txt
```

Generate the planner:

```
python semester_planner.py \
  --assignments assignments_example.csv \
  --term "Fall 2026" \
  --timezone "America/Chicago" \
  --output planner_output
```

Optionally produce a starter native spreadsheet map from an existing XLSX file:

```
python semester_planner.py \
  --assignments assignments.csv \
  --term "Fall 2026" \
  --timezone "America/Chicago" \
  --output planner_output \
  --import-xlsx existing_gradebook.xlsx
```

The output directory must be empty. The script refuses to overwrite existing work.

Input contract

```
course,title,due_date,points_possible,source_ref
BIO 210,Lab report,2026-10-22,100,Verified syllabus section 4
ENG 101,Essay draft,2026-10-26,50,Verified assignment brief
```

Required fields:

Field	Meaning
`course`	Course identifier or name
`title`	Assignment title
`due_date`	ISO date: `YYYY-MM-DD`
`points_possible`	Nonnegative numerical value
`source_ref`	Reference to the source to verify

What the script implements

- Validates required CSV columns and date syntax.
- Groups and sorts assignments by course and date.
- Generates a Word planning outline.
- Generates an Excel assignment table.
- Adds points-based grade-planning formulas.
- Creates a logical Drive folder plan without executing it.
- Records input/output checksums.

- Optionally extracts XLSX cell values, formulas, and cached values.
- Escapes potentially formula-like user text when writing Excel cells.

Limitations

- It does not parse arbitrary Word syllabi.
- It does not verify official deadlines.
- It does not calculate Excel formulas.
- It does not implement weighted grading, dropped scores, or late penalties.
- It does not perform live synchronization.
- Its native workbook map does not fully preserve charts, layouts, validations, or other advanced workbook features.

The workbook's entered-score percentage uses only assignments with entered results in its denominator. It remains an unofficial planning calculation, not an institutional grade.

3. Database schema for assignments and files

Use the following as a proposed relational schema. Reconcile it with the existing database before adding migrations; these names are not claims about current tables.

Ownership and course context

Table	Essential columns
`ownership_scopes`	`id`, `kind`, `owner_user_id`, `institution_id`
`workspaces`	`id`, `scope_id`, `kind`, `title`, `created_by`
`workspace_memberships`	`workspace_id`, `principal_id`, `role`, `revoked_at`
`academic_terms`	`id`, `institution_id`, `name`, `starts_on`, `ends_on`, `timezone`
`course_offerings`	`id`, `institution_id`, `term_id`, `course_code`, `section`, `title`
`course_memberships`	`offering_id`, `user_id`, `role`, `status`

An ownership scope must distinguish personal work from institution-owned course records. Do not assign private drafts to an institution merely because their owner is enrolled there.

Assignments

```
-- Logical schema draft, not a production migration.
-- Referenced identity, course, scope and storage entities require reconciliation.

CREATE TABLE assignments (
    id uuid PRIMARY KEY,
    scope_id uuid NOT NULL,
    course_offering_id uuid,
    title text NOT NULL,
```

```
    authority text NOT NULL CHECK (
        authority IN ('official', 'student_created', 'import_candidate')
    ),
    publication_state text NOT NULL CHECK (
        publication_state IN ('draft', 'published', 'archived')
    ),
    created_by uuid NOT NULL,
    created_at timestamptz NOT NULL DEFAULT now()
);

CREATE TABLE assignment_versions (
    id uuid PRIMARY KEY,
    assignment_id uuid NOT NULL REFERENCES assignments(id),
    version_number integer NOT NULL CHECK (version_number > 0),
    instructions jsonb NOT NULL,
    source_version_id uuid,
    rubric_version_id uuid,
    ai_policy_version_id uuid,
    published_at timestamptz,
    UNIQUE (assignment_id, version_number)
);

CREATE TABLE assignment_deadlines (
    id uuid PRIMARY KEY,
    assignment_version_id uuid NOT NULL REFERENCES assignment_versions(id),
    learner_id uuid,
    due_at timestamptz,
    due_on date,
    timezone text NOT NULL,
    verification_state text NOT NULL CHECK (
        verification_state IN ('candidate', 'verified', 'conflict')
    ),
    source_version_id uuid,
    CHECK (
        (due_at IS NOT NULL AND due_on IS NULL) OR
        (due_at IS NULL AND due_on IS NOT NULL)
    )
);

CREATE TABLE learner_assignments (
    id uuid PRIMARY KEY,
    assignment_id uuid NOT NULL REFERENCES assignments(id),
    learner_id uuid NOT NULL,
    working_state text NOT NULL CHECK (
        working_state IN (
            'not_started', 'working', 'review',
            'ready_to_submit', 'submitted'
        )
    ),
    UNIQUE (assignment_id, learner_id)
);
```

The deadline model preserves date-only deadlines without inventing a precise time. `timezone` still provides the local context.

Group assignments need a separate group-ownership model and membership checks; do not overload `learner_id` with both user and group IDs.

Files and native artifacts

```
CREATE TABLE artifacts (
    id uuid PRIMARY KEY,
    workspace_id uuid NOT NULL,
    kind text NOT NULL CHECK (
        kind IN ('document', 'workbook', 'presentation', 'binary', 'dataset')
    ),
    title text NOT NULL,
    classification text NOT NULL,
    origin text NOT NULL CHECK (
        origin IN ('native', 'imported', 'external_reference')
    ),
    created_by uuid NOT NULL,
    deleted_at timestamptz
);

CREATE TABLE artifact_versions (
    id uuid PRIMARY KEY,
    artifact_id uuid NOT NULL REFERENCES artifacts(id),
    version_number integer NOT NULL CHECK (version_number > 0),
    schema_version integer NOT NULL,
    storage_ref text NOT NULL,
    sha256 text NOT NULL CHECK (length(sha256) = 64),
    source_version_id uuid,
    created_by uuid NOT NULL,
    created_at timestamptz NOT NULL DEFAULT now(),
    UNIQUE (artifact_id, version_number)
);

CREATE TABLE assignment_artifacts (
    learner_assignment_id uuid NOT NULL REFERENCES learner_assignments(id),
    artifact_id uuid NOT NULL REFERENCES artifacts(id),
    purpose text NOT NULL CHECK (
        purpose IN ('notes', 'source', 'working_draft', 'deliverable')
    ),
    PRIMARY KEY (learner_assignment_id, artifact_id, purpose)
);
```

Store large document bodies and binaries in controlled storage, with database references and version metadata. Submitted versions must be immutable.

Additional required tables

Table	Purpose
`access_grants`	Explicit read, comment, edit, or publish permission
`source_versions`	Exact imported/retrieved source and checksum
`import_jobs`	Input, mapping configuration, warnings, result

Table	Purpose
`workbook_versions`	Native workbook metadata and calculation-engine version
`submissions`	Destination, result, receipt, status
`submission_items`	Exact artifact versions submitted
`integration_connections`	Provider, approved scope, vault reference
`external_object_links`	External ID ↔ Semester artifact mapping
`sync_cursors`	Durable provider change cursor
`sync_jobs`	Retryable work with idempotency
`sync_conflicts`	Base/native/external version differences
`audit_events`	Authorized action metadata
`outbox_events`	Transactionally recorded downstream work

Security requirements before activation

The SQL above deliberately does not include ready-to-run authorization policies. Production migrations must also implement:

- Foreign keys to the reconciled canonical entities.
- Scope-consistent relationships.
- Row-level security and capability checks.
- Storage authorization.
- Revocation behavior.
- Immutable published/submitted versions.
- Retention and deletion rules.
- Restricted function ownership and execution grants.
- Tenant isolation tests.

The dated project architecture specifically reports a risk from privileged function owners bypassing RLS. Adding tables with RLS alone would not resolve that function-level risk. [1]

4. Excel import and native spreadsheet mapping

Two separate import modes

Workbook import

Preserve the spreadsheet as an editable native workbook.

Assignment-table import

Extract selected rows into assignment candidates using an explicit column mapping.

Do not assume every row in a gradebook is an assignment or that every date is an official deadline.

Native cell mapping

Excel feature	Native representation	Required handling
Worksheet	Sheet with stable ID	Preserve title and order
Cell address	Coordinate plus stable identity where needed	Track structural edits
Text	String value	Preserve literal text
Number	Numerical value	Preserve formatting separately
Boolean	Boolean value	Do not convert to display text
Date/time	Typed date/time plus workbook date-system metadata	Avoid implicit timezone conversion
Formula	Formula text and dependency metadata	Evaluate only supported functions
Cached formula result	Imported cached value	Mark freshness as unknown
Error	Typed spreadsheet error	Do not silently replace with zero
Number format	Formatting metadata	Keep distinct from value
Merged cells	Explicit merged range	Preserve or warn
Named ranges	Workbook/sheet range definition	Resolve references
Chart	Chart specification	Convert supported charts; otherwise flag
External links/macros	Unsupported or quarantined feature	No execution
Validation	Constraint definition	Convert supported rules or warn

`openpyxl` distinguishes formula text from stored cached values through `data_only`. Those cached values are what was stored when the workbook was last calculated; they are not newly computed results. VBA preservation also does not mean VBA execution is supported.[2] [3]

Example native cell

```
{
  "address": "E12",
  "type": "formula",
  "formula": "=SUM(E2:E11)",
  "cached_value": 84,
  "cached_value_freshness": "unknown",
```

```
  "calculation_state": "pending_native_evaluation",
  "number_format": "0.00"
}
```

The supplied script implements a smaller extraction format and marks formulas `not_evaluated`. It does not implement the full mapping above.

Assignment column mapping

```
{
  "sheet": "Assignments",
  "header_row": 1,
  "columns": {
    "course": "A",
    "title": "B",
    "due_date": "C",
    "points_possible": "D",
    "source_ref": "G"
  },
  "authority": "import_candidate",
  "date_policy": "preserve_date_only",
  "duplicate_policy": "review"
}
```

Import pipeline

1. Validate file type and decompressed-size limits.
2. Preserve the original file and checksum.
3. Inspect sheets and unsupported features.
4. Ask the user to select workbook import or assignment extraction.
5. Preview the mapped content.
6. Parse types and validate dates.
7. Flag ambiguous rows and duplicate candidates.
8. Commit only the approved mapping.
9. Preserve row/cell provenance.
10. Generate a reconciliation report.

For production, add malformed-file handling, archive expansion limits, formula compatibility tests, workbook date-system tests, and permission-aware storage.

5. Google Drive API integration map

Supported modes

Mode	Semester behavior
Reference	Keep an external link and authorized metadata
Import	Create a native copy from a particular external version
Managed export	Write an approved Semester version to a resolved Drive target
Bidirectional sync	Later activation after conflict and conversion evidence

Begin with references, imports, and managed exports. Generic bidirectional editing is a separate capability—not an automatic consequence of polling Drive.

API mapping

Operation	Drive API method	Semester responsibility
Inspect selected file	`files.get`	Check MIME type, access, version metadata
List authorized contents	`files.list`	Paginate and constrain selection scope
Download binary DOCX/XLSX/PPTX	`files.get` with media download	Preserve source checksum
Export Google-native document	`files.export`	Request a supported export MIME type
Create approved folder	`files.create`	Use folder MIME type and resolved parent
Upload new artifact	`files.create`	Record returned file ID
Update managed target	`files.update`	Verify target and detect conflicting changes
Initialize changes cursor	`changes.getStartPageToken`	Persist cursor for the connection/scope
Read changes	`changes.list`	Process pages durably
Subscribe to changes	`changes.watch`	Validate channel and schedule reconciliation
Stop channel	`channels.stop`	Clean up revoked/expired connections

Google's Drive v3 reference provides the file create/export and changes endpoints; its Python client also exposes file upload, export, and paginated listing methods. Drive supports resource-change notification channels, including subscriptions through `changes.watch`. [4] [5] [6] [7]

MIME-type handling

Source	Import route
Uploaded `.docx`	Download binary → Word importer
Uploaded `.xlsx`	Download binary → workbook importer
Uploaded `.pptx`	Download binary → presentation importer
Google Docs	Export to an appropriate supported format → importer
Google Sheets	Export to an appropriate supported format → importer

Source	Import route
Google Slides	Export to an appropriate supported format → importer
Folder	Metadata/container mapping; no document import

Google-native documents use the export API rather than the same binary-download path used for uploaded Office files.[5] [4]

Authorization

Prefer user-selected file access with `drive.file` where that scope covers the intended workflow. Do not request full-Drive access merely to simplify discovery. Google documents `drive.file` as access to files created by the app or explicitly opened/shared with it.[8]

Credentials belong in a server-side vault. The browser and AI layer should receive neither refresh tokens nor service credentials.

Reliable change processing

```
Drive notification
    ↓
Validate registered channel
    ↓
Queue reconciliation
    ↓
Read changes from durable cursor
    ↓
Filter to approved mapped objects
    ↓
Check access and external version
    ↓
Import, export, or create conflict
    ↓
Commit result and cursor
    ↓
Update dashboard sync state
```

Use notifications as triggers to reconcile, not as proof that content has been synchronized.

Keep periodic reconciliation as a recovery mechanism. Access loss should restrict the external object immediately; retention of an earlier imported copy requires a separate applicable policy.

Conflict contract

When both sides changed after the last common version:

- Keep the local version.
- Keep the external version.
- Record the last common version.

- Show the conflict.
- Offer an explicit reviewed resolution.
- Never silently overwrite graded work or lecture notes.

Useful dashboard states:

`up_to_date`, `pending_import`, `pending_export`, `conflict`, `access_revoked`, `retrying`, `failed`.

“Last synchronized at” must reflect a completed reconciliation, not the arrival of a webhook.

6. Approval and execution boundary

The generated Drive folder plan contains logical folder paths only. Before executing it:

1. Resolve the Google account and actual parent folder ID.
2. Preview every folder and file operation.
3. Check existing mappings and duplicate names.
4. Request approval for the exact resolved changes.
5. Execute approved operations.
6. Record returned IDs and receipts.
7. Reconcile before retrying uncertain outcomes.

The planner’s output manifest explicitly records that no external actions were executed.

7. Verification completed and next release gates

Completed here

- Generated the Python script and supporting files.
- Ran the script with two example assignments.
- Verified a successful process exit.
- Verified creation of the DOCX, XLSX, folder plan, and checksum manifest.

This is a local generation smoke test, not evidence of Office rendering fidelity, live Google Drive sync, a database migration, or production readiness.

Required before dashboard activation

Gate	Acceptance evidence
Assignment import	Duplicate and ambiguous-date cases handled
Native spreadsheet	Supported formulas match reference fixtures
Word notes	Supported structure and source references preserved
Drive authorization	Revoked access and narrow scopes tested
Sync reliability	Pagination, retry, missed-event recovery, conflicts tested

Gate	Acceptance evidence
Data isolation	Personal and institutional boundaries verified
Submission	Exact versions and verified receipts preserved
Accessibility	Dashboard and editor keyboard/screen-reader workflows tested
Recovery	Original imports, previous versions, and rollback demonstrated

The immediate build slice is the complete path from selected source → reviewed assignment import → native document/workbook → dashboard tracking → approved Drive export. That gives Semester a useful multi-app workflow without falsely claiming autonomous synchronization or official academic authority.

⁂

1. The Main Semester design system (2) copy/uploads/Semester build Architecture.pdf
2. https://openpyxl.readthedocs.io/en/3.1/api/openpyxl.reader.excel.html
3. https://openpyxl.readthedocs.io/en/3.1/tutorial.html
4. https://developers.google.com/workspace/drive/api/reference/rest/v3
5. https://googleapis.github.io/google-api-python-client/docs/dyn/drive_v3.files.html
6. https://developers.google.com/workspace/drive/api/guides/push
7. https://developers.google.com/workspace/drive/api/reference/rest/v3/changes/watch
8. https://developers.google.com/workspace/drive/api/guides/api-specific-auth
9. https://openpyxl.readthedocs.io/en/3.1/_modules/openpyxl/reader/excel.html
10. https://googleapis.dev/java/google-api-services-drive/latest/com/google/api/services/drive/Drive.Files.Create.html
11. https://openpyxl.readthedocs.io/en/3.1/formula.html
12. https://stackoverflow.com/questions/28517508/how-can-i-use-openpyxl-to-read-an-excel-cell-value-and-not-the-formula-computing
13. https://www.fragments-studio.com/en/integrations/google-drive
14. https://tuxpool.blogspot.com/2025/03/openpyxl-dataonly-beware.html
15. https://www.reddit.com/r/learnpython/comments/fmgz20/the_pain_of_using_openpyxl/
16. https://www.emptor.io/blog/demystifying-the-google-drive-changes-api
17. https://fast.io/resources/google-drive-api-list-files/
18. https://discuss.google.dev/t/visibility-of-files-and-folder-using-the-google-drive-api/328507
19. https://openpyxl.readthedocs.io/en/2.4/usage.html